

Functional Programming Report Layer

Table of Contents

- Programación funcional e interfaces funcionales en reportes
 - Resultado verificado
 - Interfaces funcionales implementadas
 - Mapa para sustentación: concepto → código → prueba → demostración
 - Reglas de negocio que calcula la capa funcional
 - Asistencia
 - Finanzas
 - Operación académica
 - Flujo real de ejecución
 - Evidencia ejecutable
 - Qué afirmar y qué no afirmar
 - Conclusión

Programación funcional e interfaces funcionales en reportes

Resultado verificado

La implementación funcional está conectada a los reportes reales de American Latin Class y tiene pruebas automatizadas. La capa principal está en `06Code/src/functional/reportMetrics.js`; `ReportsController` obtiene únicamente los datos autorizados y finalizados, y después inyecta esos datos y una fecha de corte en las funciones de cálculo.

Esta separación es deliberada:

- `ReportsController`: autenticación, alcance por sede, consulta de repositorios, filtro del periodo y respuesta HTTP.
- `reportMetrics.js`: transformación determinista de datos, cálculos y composición del reporte.
- `RulesService` y `AttendanceService`: decisiones individuales de beca, promoción y pago docente que sí necesitan repositorios.

No se debe decir que todo el sistema es funcional. El sistema usa programación orientada a objetos para controladores y servicios, y una **capa funcional** para los cálculos de reportes. Esa es una arquitectura híbrida válida.

Interfaces funcionales implementadas

JavaScript no tiene la anotación `@FunctionalInterface` de Java. En este proyecto el contrato de una función se expresa con JSDoc y se usa mediante funciones de primera clase:

```
/** @template T @typedef {(item: T) => boolean} Predicate */
/** @template T @template R @typedef {(item: T) => R} Mapper */
/** @template T @template R @typedef {(accumulator: R, item: T) => R}
    Reducer */
/** @template T @typedef {(item: T) => number|string|null|undefined}
    NumberSelector */
/** @template TContext @template TResult
    * @typedef {(context: TContext) => TResult} ReportMetricCalculator
    */
```

JSDoc documenta y permite que el editor compruebe el contrato; no crea validación de tipos en tiempo de ejecución. La evidencia funcional está en que esas funciones se reciben como parámetros y se ejecutan, por ejemplo:

```
sumBy(payments, (payment) => payment.amount);
countBy(students, (student) => student.active !== false);
idSetFrom(classGroups, (group) => group.id);
branches.reduce(combineBranchTotals, emptyTotals());
```

Mapa para sustentación: concepto → código → prueba → demostración

Concepto	Función o fragmento real	Prueba automatizada	Qué mostrar en la demo
Función de orden superior	<code>sumBy(items, selector)</code>	<code>functional helpers receive behaviors as parameters</code>	Cambiar el selector de <code>amount</code> a otro campo sin modificar <code>sumBy</code>
<code>Predicate<T></code>	<code>countBy(items, predicate)</code>	La misma prueba de helpers	Mostrar que el predicado de pago pagado devuelve un conteo
<code>Mapper<T,R></code>	<code>idSetFrom(items, selector)</code>	La misma prueba de helpers	Construir IDs de estudiantes con <code>(payment) => payment.studentId</code>
<code>Reducer<T,R></code>	<code>combineBranchTotals</code> y <code>reduce</code>	<code>academic report composes branch metrics into general totals</code>	Señalar cómo varias sedes producen un consolidado
<code>NumberSelector<T></code>	Parámetro <code>selector</code> de <code>sumBy</code>	Prueba de helpers y métricas financieras	Sumar mensualidades, otros ingresos o eventos con el mismo helper
<code>ReportMetricCalculator</code>	<code>calculateBranchStudentMetrics</code>	Pruebas de reporte por sede	Mostrar que recibe contexto y devuelve una métrica, sin consultar la BD
Función pura	<code>calculateAttendanceMetrics</code>	Prueba de ausencia justificada	Ejecutarla dos veces con los mismos datos y obtener el mismo resultado
Inmutabilidad	<code>buildAcademicReport</code> crea arreglos/objetos nuevos	<code>report functions do not mutate the input source</code>	Mostrar el <code>snapshot</code> antes y después
Composición	<code>buildAcademicReport</code> → <code>createBranchReport</code> → métricas	Prueba del reporte académico general	Seguir el flujo desde la función general a las métricas pequeñas

Concepto	Función o fragmento real	Prueba automatizada	Qué mostrar en la demo
Reloj inyectado	<code>referenceAt</code> , <code>referenceDate</code> , <code>ref</code>	Pruebas con <code>2026-07-10T12:00:00.000Z</code>	Explicar por qué una fecha fija hace el resultado reproducible
Regla de negocio aislada	<code>calculateOccupancyMetrics</code>	Prueba financiera, ocupación y embudo	Mostrar capacidad, matrículas activas y lista de espera
Calidad auditable	<code>buildQualityAlerts</code>	Dos pruebas de alertas	Mostrar códigos estables como <code>UNFINALIZED_ATTENDANCE</code>

Reglas de negocio que calcula la capa funcional

Asistencia

- Presencia física: únicamente `present` y `late` cuentan como asistencia.
- Una ausencia justificada **sigue siendo ausencia** en el indicador crudo.
- Si la justificación fue aprobada, se excluye del denominador del indicador ajustado.
- `rawAttendanceRate` muestra lo ocurrido físicamente.
- `attendanceRate` muestra el indicador ajustado.
- `punctualityRate` divide llegadas puntuales entre asistencias físicas.
- El controlador entrega a la capa funcional solo sesiones no canceladas con asistencia `finalized`.

Ejemplo probado: dos asistencias, dos ausencias y una excusa aprobada producen 50 % crudo y 66,67 % ajustado.

Finanzas

- Se separan mensualidades de otros conceptos.
- Solo movimientos `paid` se reconocen como recaudo.
- Los reversos se conservan como movimientos negativos auditables.
- Se muestran recaudo bruto, valor reversado y recaudo neto.
- Obligaciones `pending` y `overdue` forman la cartera; el vencimiento se deriva respecto a una fecha de corte.
- La cartera se distribuye en rangos de antigüedad.
- Los eventos futuros no se reconocen antes de la fecha de corte.

Operación académica

- La ocupación usa matrículas `active` o `trial` vigentes.
- La lista de espera se informa por separado.
- La capacidad disponible nunca se presenta como negativa.
- El embudo normaliza prospectos nuevos, contactados, con clase de prueba, matriculados y perdidos.
- Las alertas de calidad detectan, entre otros casos, pagos sin fecha, asistencias no finalizadas, cobros duplicados, grupos sobrecupo y referencias huérfanas.

Flujo real de ejecución



La fecha actual se obtiene en el controlador, que es la frontera con el mundo externo. La capa funcional recibe `generatedAt` y `asOf`. En las pruebas se inyecta una fecha fija; así los cálculos sensibles al tiempo son reproducibles.

Evidencia ejecutable

Desde `06Code`:

```
npm test -- --runInBand --runTestsByPath tests/unit/report-
metrics.test.js
```

Resultado verificado el 13 de julio de 2026:

```
PASS tests/unit/report-metrics.test.js
Test Suites: 1 passed, 1 total
```

Tests: 9 passed, 9 total

Para probar las reglas conectadas de beca, promoción, justificación y pago docente:

```
npm test -- --runInBand --runTestsByPath tests/unit/rules.test.js
```

Para validar el recorrido HTTP y las reglas operativas del reporte:

```
npm test -- --runInBand --runTestsByPath tests/unit/report-  
metrics.test.js tests/unit/rules.test.js  
tests/integration/operational-integrity.test.js  
tests/integration/academic.test.js
```

Resultado focal completo verificado el 13 de julio de 2026:

Test Suites: 4 passed, 4 total
Tests: 31 passed, 31 total

Qué afirmar y qué no afirmar

Sí se puede afirmar:

- Hay funciones de orden superior reales, no ejemplos aislados.
- Existen contratos funcionales JSDoc y usos concretos como callbacks.
- El núcleo de cálculo no accede a Express ni a la base de datos.
- Con datos y fecha de corte iguales, el reporte produce el mismo resultado.
- La prueba de inmutabilidad demuestra que la fuente no se modifica.
- Los resultados llegan a endpoints y gráficos reales del sistema.

No se debe afirmar:

- Que JSDoc convierte JavaScript en un lenguaje con interfaces de tiempo de ejecución.
- Que una ausencia justificada equivale a presencia.
- Que `RulesService` es una clase pura: consulta repositorios y pertenece a la frontera de aplicación.
- Que usar `map` o `reduce` por sí solo vuelve funcional a todo el programa.

Conclusión

La implementación cumple el objetivo académico y aporta valor al producto: el código recibe comportamientos como parámetros, compone funciones pequeñas, evita mutar entradas, inyecta la fecha de corte y separa el cálculo de

los efectos externos. Gracias a ello, los indicadores que usa el director son repetibles, probables y auditables.